

Introduction to Programming

Problem Sheet 2

Recursion Practice Problem Set All functions must be implemented using recursion (no built-in iteration). Use only Integer (no Int), no lists, only tuples where necessary. No imports

1. Factorial of a non-negative integer.

Compute $n! = 1 \times 2 \times \dots \times n$.

```
factorial :: Integer -> Integer
factorial = undefined
```

2. Power: raise x to the n-th power ($n \geq 0$).

Compute x^n using only multiplication and recursion.

```
power :: Integer -> Integer -> Integer
power = undefined
```

3. Sum of digits of a non-negative integer.

e.g., `sumDigits 123 = 6`.

```
sumDigits :: Integer -> Integer
sumDigits = undefined
```

4. Product of digits (non-negative integer).

e.g., `productDigits 234 = 24`.

```
productDigits :: Integer -> Integer
productDigits = undefined
```

5. Sum of integers from a to b (inclusive), assuming $a \leq b$.

```
sumRange :: Integer -> Integer -> Integer
sumRange = undefined
```

6. Count the number of digits in a non-negative integer.

e.g., `countDigits 0 = 1`, `countDigits 123 = 3`.

```
countDigits :: Integer -> Integer
countDigits = undefined
```

7. Check if a non-negative integer is even.

Use only recursion (no ``mod``, ``even``, or division).

```
isEven :: Integer -> Bool
isEven = undefined
```

8. Compute the nth triangular number: $T(n) = 1 + 2 + \dots + n$.

```
triangular :: Integer -> Integer
triangular = undefined
```

9. Fibonacci number (naïve double recursion).

Define $\text{fib}(0) = 0$, $\text{fib}(1) = 1$, and for $n > 1$ use the sum of the two previous.

```
fib :: Integer -> Integer
fib = undefined
```

10. Binomial coefficient $C(n,k)$ using Pascal's rule.

$C(n,k)$ counts the number of ways to choose k items from n .

```
binom :: Integer -> Integer -> Integer
binom = undefined
```

11. Convert a number from one base to another.

Given n (written in base b_1) and target base b_2 ($2 \leq b_1, b_2 \leq 10$), return the representation of the same value in base b_2 as an Integer whose decimal digits are the digits of the new representation.
e.g., $\text{baseConvert } 123 \ 4 \ 8 = 33$ (because $123_4 = 27_{10} = 33_8$)

```
baseConvert :: Integer -> Integer -> Integer -> Integer
baseConvert = undefined
```

13. Power tower of height n with base n .

Compute $n^{(n^{(\dots^n)})}$ where the tower has n occurrences of n .
This function grows extremely fast using only multiplication and recursion.

```
powTow :: Integer -> Integer
powTow = undefined
```

14. Sum of two integers using only increment and decrement operations.

(No use of $+$, only successor and predecessor.)

```
addRec :: Integer -> Integer -> Integer
addRec = undefined
```

15. Multiply two integers using only addition and recursion.

Assume non-negative inputs.

```
mulRec :: Integer -> Integer -> Integer
mulRec = undefined
```

16. Apply a function n times to an initial value.

e.g., `iterate' 3 succ 0 = 3`.

```
iterate' :: Integer -> (Integer -> Integer) -> Integer -> Integer
iterate' = undefined
```

17. Repeatedly apply a function until a condition holds.

Given f and p , keep applying f while p is false,
return the first value satisfying p . (Assume termination.)

```
until' :: (Integer -> Bool) -> (Integer -> Integer) -> Integer -> Integer
until' = undefined
```